LAPORAN PRAKTIKUM 6
PEMOGRAMAN BERORIENTASI OBJEK
ABSTRACTION



Disusun Oleh:

Annisa Layli Ramadhani

2511532024

Dosen Pengampu:

Afdhal Dinilhak, M.Kom

Asisten Praktikum:

Muhammad Dawi Syauqi

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

A. Identitas

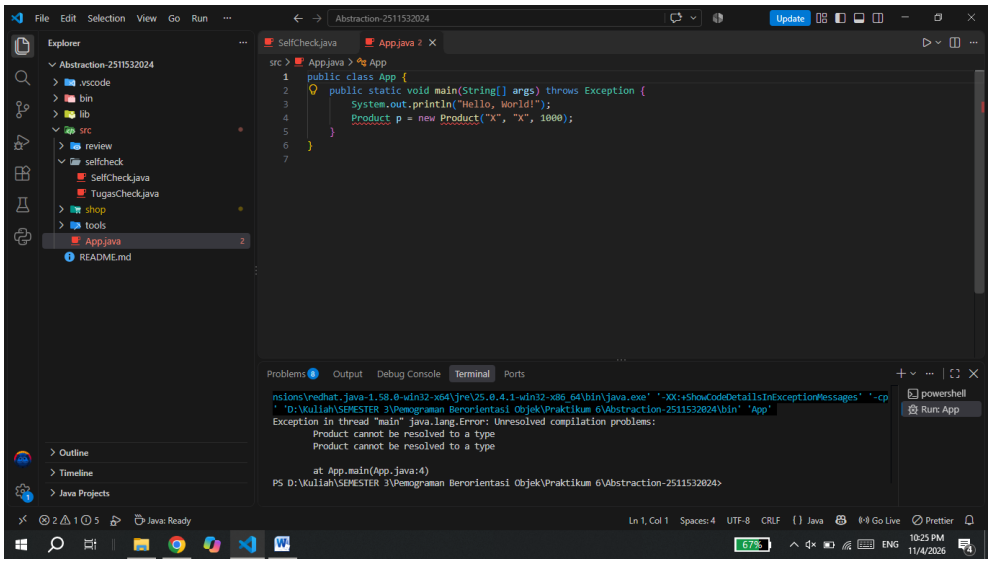
Nama	Annisa Layli Ramadhani
NIM	2511532024
Kelas	Informatika B

Tools.NimParams

```
Parameter tugas untuk NIM 2511532024
  CARD_ADMIN_PERCENT  = 2
  TRANSFER_ADMIN_FEE  = 4500
  SMS_MAX_LENGTH      = 124
```

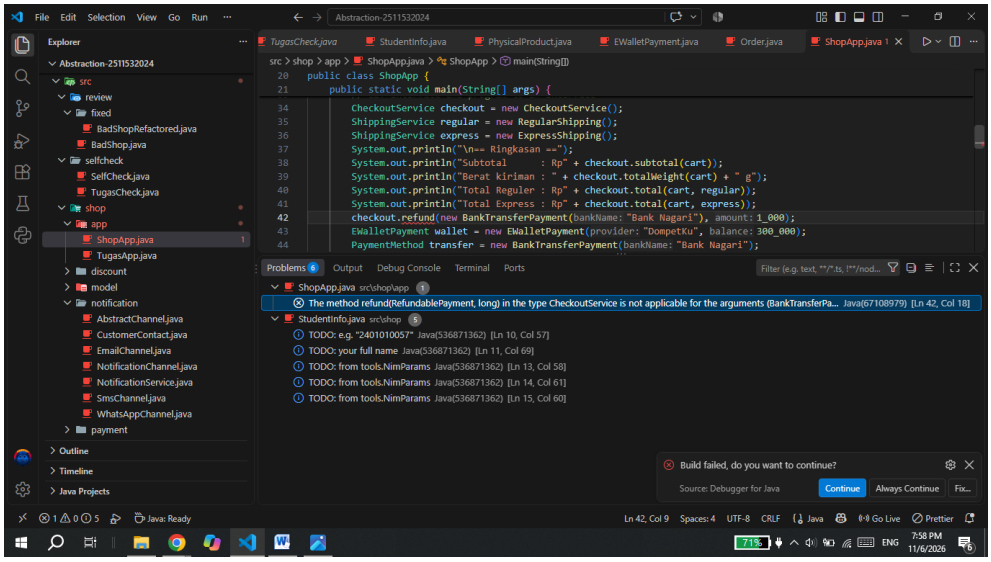
B. Tangkapan Layar *Error Compiler*

1. L1



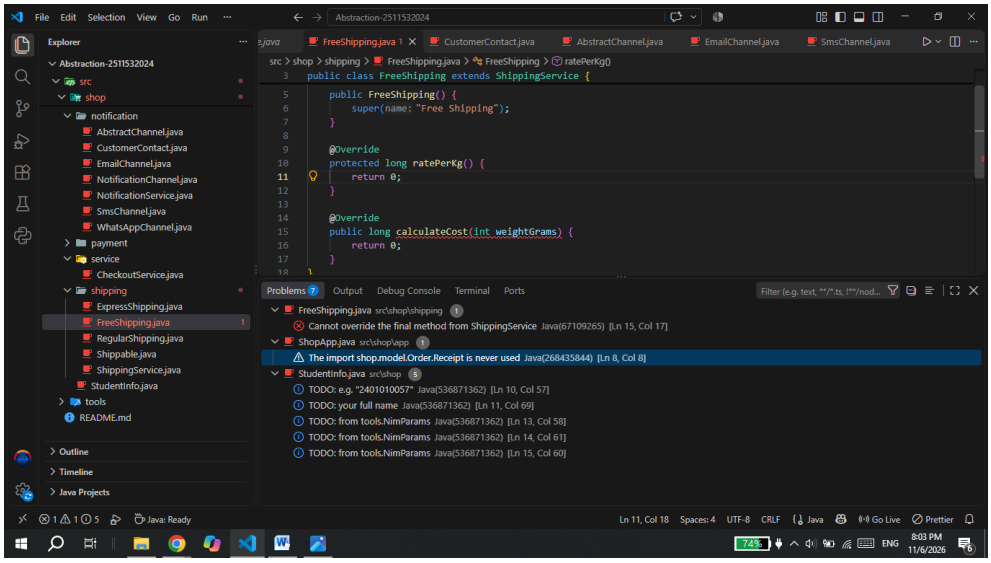
Compiler menolak *new Product(...)* karena *Product* adalah *abstract class*, yang berfungsi sebagai cetakan bersama dan tidak boleh dibuat objeknya langsung. Objek hanya bisa dibuat dari subclass konkret seperti *PhysicalProduct* atau *DigitalProduct.*

2. L3



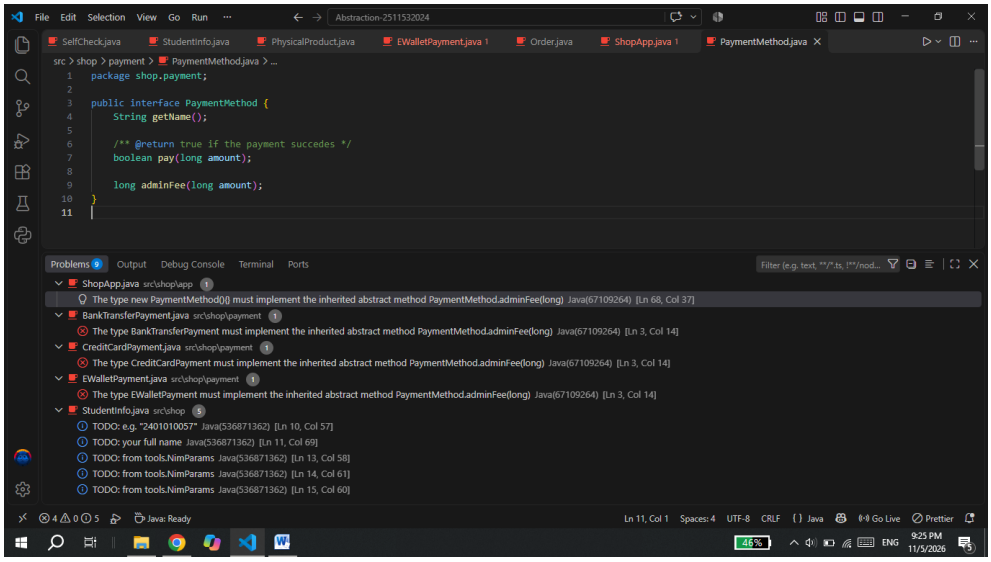
CheckoutService.refund hanya menerima *RefundablePayment*, sedangkan *BankTransferPayment* hanya meng-implement *PaymentMethod. Compiler* menolak *refund* lewat transfer bank sebelum program berjalan, tanpa perlu *if* apa pun.

3. L4



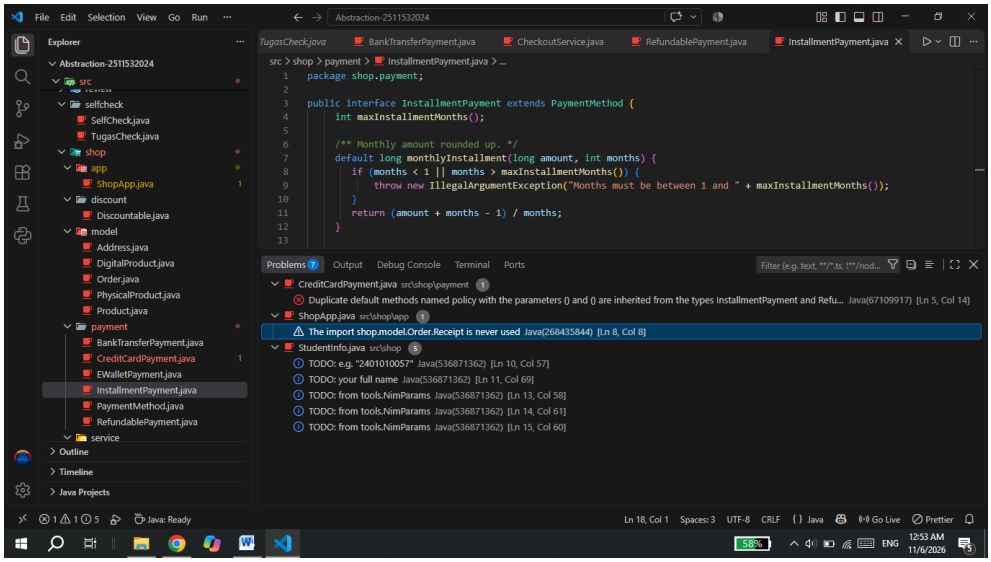
calculateCost di *ShippingService* bersifat *final*, sehingga *FreeShipping* tidak boleh meng-*override*-nya. Rumus ongkir terkunci, sedangkan *subclass* tetap bebas mengatur tarif lewat *ratePerKg()*.

4. **T1.a**



Menambah *method abstract adminFee* ke *PaymentMethod* membuat 4 class yang meng-implement-nya *error:* tiga *class* pembayaran dan *anonymous class voucher* di *ShopApp*. Setiap *class* yang meng-implement *interface* wajib mengisi semua *method abstract*-nya, dan *anonymous class* juga terikat aturan itu.

5. **T1.e**



CreditCardPayment meng-implement dua *interface* yang sama-sama punya *default method policy()* dengan isi berbeda, sehingga Java tidak tahu versi mana yang harus dipakai. *Class* itu wajib meng-*override*

policy() dan memilih sendiri, misalnya dengan *RefundablePayment.super.policy()* dan *InstallmentPayment.super.policy()*.

C. Penjelasan Kode Program

1. Latihan

a. L1 – *Abstract class* dan *abstract method*

1) L1.a – *Product*

```
package shop.model;

public abstract class Product {
    private final String sku;
    private final String name;
    private final long price;

    protected Product(String sku, String name, long price)
{
        if (sku == null || sku.isBlank()) {
            throw new IllegalArgumentException("SKU must
not be blank");
        }
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must
not be blank");
        }
        if (price < 0) {
            throw new IllegalArgumentException("Price must
not be negative");
        }
        this.sku = sku;
        this.name = name;
        this.price = price;
    }

    public String getSKU() {
        return sku;
    }
    public String getName() {
        return name;
    }
    public long getPrice() {
        return price;
    }
```

```
    /** Price the customer actually pays. Subclasses may
override it. */
    public long getFinalPrice() {
        return price;
    }
    /** Every concrete product must state its category. */
    public abstract String category();

    /** A concrete method that relies on the abstract
method category(). */
    public String getInfo() {
        return "[" + category() + "] " + name + " - Rp" +
getFinalPrice();
    }
}
```

Penjelasan:

Class Product merupakan *abstract class* yang berfungsi sebagai *class* induk untuk berbagai jenis produk pada aplikasi toko. *Class* ini memiliki tiga atribut utama, yaitu *sku, name,* dan *price* yang digunakan untuk menyimpan kode produk, nama produk, dan harga asli. Ketiga atribut tersebut menggunakan *modifier private* dan *final* sehingga nilainya hanya dapat diatur melalui *constructor* dan tidak dapat diubah setelah objek dibuat.

Class ini menerapkan konsep *abstraction* dengan menyediakan *method abstrak category()* yang wajib diimplementasikan oleh setiap *subclass.* Selain itu, terdapat beberapa *method* konkret seperti *getSKU(), getName(), getPrice(), getFinalPrice(),* dan *getInfo()* untuk mengambil serta menampilkan informasi produk. Dengan adanya *class Product*, setiap jenis produk dapat memiliki karakteristik dan implementasi masing-masing tanpa harus membuat ulang atribut dan method dasar yang sama.

2) L1.b – Class *PhysicalProduct*

```
package shop.model;

public class PhysicalProduct extends Product {
    private final int weightGrams;
```

```
    public PhysicalProduct(String sku, String name, long
price, int weightGrams) {
        super(sku, name, price);
        if (weightGrams <= 0) {
            throw new IllegalArgumentException("Weight
must not be negative");
        }
        this.weightGrams = weightGrams;
    }

    @Override
    public String category() {
        return "Fisik";
    }
}
```

Penjelasan:

Class PhysicalProduct merupakan *subclass* dari *class Product* yang digunakan untuk merepresentasikan produk fisik atau produk yang memiliki bentuk dan berat. *Class* ini mewarisi atribut dan *method* dari *class Product*, kemudian menambahkan atribut *weightGrams* untuk menyimpan berat produk dalam satuan gram. *Constructor* menggunakan *super()* untuk menginisialisasi atribut dasar dari class induk serta melakukan validasi terhadap berat produk agar nilainya lebih dari nol. *Class* ini juga mengimplementasikan method abstrak *category()* dengan mengembalikan nilai *"Fisik"* sebagai kategori produk. Dengan demikian, *class PhysicalProduct* menerapkan konsep *inheritance* dan *abstraction* dalam pemrograman berorientasi objek.

3) L1.c – *Class DigitalProduct*

```
package shop.model;

public class DigitalProduct extends Product {
    private final String downloadUrl;

    public DigitalProduct(String sku, String name, long
price, String downloadUrl) {
        super(sku, name, price);
```

```
        if (downloadUrl == null || downloadUrl.isBlank())
{
            throw new IllegalArgumentException("Download
URL must not be blank");
        }
        this.downloadUrl = downloadUrl;
    }

    public String getDownloadUrl() {
        return downloadUrl;
    }

    @Override
    public String category() {
        return "Digital";
    }
}
```

Penjelasan:

Class DigitalProduct merupakan *subclass* dari *class Product* yang digunakan untuk merepresentasikan produk digital, yaitu produk yang dapat diakses atau diunduh secara digital. *Class* ini mewarisi atribut dan *method* dari *class Product*, kemudian menambahkan atribut *downloadUrl* untuk menyimpan alamat URL unduhan produk. *Constructor* menggunakan *super()* untuk menginisialisasi atribut dasar dari *class* induk serta melakukan validasi agar URL unduhan tidak kosong. *Method getDownloadUrl()* digunakan untuk mengambil alamat URL unduhan, sedangkan *method category()* mengimplementasikan *method* abstrak dari *class Product* dengan mengembalikan nilai *"Digital"*. *Class* ini menerapkan konsep *inheritance* dan *abstraction* dalam pemrograman berorientasi objek.

4) L1.d – *Class ShippingService*

```
package shop.shipping;

public abstract class ShippingService {
    private final String name;
    protected ShippingService(String name) {
        if (name == null || name.isBlank()) {
```

```
            throw new IllegalArgumentException("Name must
not be blank");
        }
        this.name = name;
    }

    public String getName() {
        return name;
    }

    /** Rate in rupiah per started kilogram. */
    protected abstract long ratePerKg();

    /** Weight is rounded up: 700 g -> 1 kg, 1001 g -> 2
kg. */
    public long calculateCost(int weightGrams) {
        if (weightGrams <= 0) {
            return 0;
        }
        long kilograms = (weightGrams + 999L) / 1000;
        return kilograms * ratePerKg();
    }
}
```

Penjelasan:

Class ShippingService merupakan *abstract class* yang digunakan sebagai kelas dasar untuk berbagai jenis layanan pengiriman. *Class* ini memiliki atribut name untuk menyimpan nama layanan pengiriman yang nilainya diinisialisasi melalui *constructor* dan divalidasi agar tidak kosong. *Method getName()* digunakan untuk mengambil nama layanan, sedangkan *method* abstrak *ratePerKg()* wajib diimplementasikan oleh setiap subclass untuk menentukan tarif pengiriman per kilogram. Selain itu, *method calculateCost()* digunakan untuk menghitung biaya pengiriman berdasarkan berat produk dalam gram. Perhitungan berat menggunakan pembulatan ke atas, sehingga berat 700 gram dihitung sebagai 1 kilogram dan berat 1.001 gram dihitung sebagai 2 kilogram. Dengan demikian, *class* ini menerapkan konsep *abstraction* dan *inheritance* untuk menyediakan dasar perhitungan biaya pada berbagai jenis layanan pengiriman.

5) **L1.e – *Class RegularShipping* dan *ExpressShipping***

RegularShipping

```
package shop.shipping;

public class RegularShipping extends ShippingService {
    public RegularShipping() {
        super("Reguler");
    }

    @Override
    protected long ratePerKg() {
        return 10_000;
    }
}
```

Penjelasan:

Class RegularShipping merupakan *subclass* dari *class ShippingService* yang digunakan untuk merepresentasikan layanan pengiriman reguler. *Class* ini mewarisi atribut dan *method* dari *class* induknya. Pada *constructor*, *super("Reguler")* digunakan untuk memanggil *constructor ShippingService* sekaligus menetapkan nama layanan pengiriman menjadi *"Reguler"*. *Class* ini juga melakukan *override* terhadap *method ratePerKg()* untuk menentukan tarif pengiriman reguler sebesar Rp10.000 per kilogram. Dengan demikian, *class RegularShipping* menerapkan konsep *inheritance* dan *abstraction* karena mengimplementasikan *method* abstrak yang telah didefinisikan pada class induknya.

ExpressShipping

```
package shop.shipping;

public class ExpressShipping extends ShippingService {
    public ExpressShipping() {
        super("Express");
    }

    @Override
    protected long ratePerKg() {
        return 18_000;
    }
```

```
}
```

Penjelasan:

Class ExpressShipping merupakan *subclass* dari *class ShippingService* yang digunakan untuk merepresentasikan layanan pengiriman ekspres. *Class* ini mewarisi atribut dan *method* dari *class* induknya. Pada *constructor*, *super("Express")* digunakan untuk memanggil *constructor ShippingService* sekaligus menetapkan nama layanan pengiriman menjadi *"Express"*. *Class* ini juga melakukan *override* terhadap *method ratePerKg()* untuk menentukan tarif pengiriman ekspres sebesar Rp18.000 per kilogram. Dengan demikian, *class ExpressShipping* menerapkan konsep *inheritance* dan *abstraction* karena mengimplementasikan *method* abstrak yang telah didefinisikan pada *class* induknya.

b. L2 – *Interface* dan *program to an interface*

1) L2.a – *Shippable*

Shippable

```
package shop.shipping;

public interface Shippable {
    int getWeightGrams();
}
```

Penjelasan:

Interface Shippable merupakan *interface* yang digunakan sebagai aturan atau kontrak bagi class yang memiliki karakteristik dapat dikirim. *Interface* ini mendefinisikan *method getWeightGrams()* dengan tipe data int untuk memperoleh berat suatu produk dalam satuan gram. Setiap *class* yang mengimplementasikan *interface Shippable* wajib menyediakan implementasi dari *method* tersebut. Dengan demikian, *interface* ini menerapkan konsep *abstraction* karena hanya menentukan *method* yang harus tersedia tanpa menjelaskan detail proses implementasinya.

PhysicalProduct

```
package shop.model;

import shop.shipping.Shippable;

public class PhysicalProduct extends Product implements
Shippable{
    private final int weightGrams;

    public PhysicalProduct(String sku, String name, long
price, int weightGrams) {
        super(sku, name, price);
        if (weightGrams <= 0) {
            throw new IllegalArgumentException("Weight
must not be negative");
        }
        this.weightGrams = weightGrams;
    }

    @Override
    public int getWeightGrams() {
        return weightGrams;
    }

    @Override
    public String category() {
        return "Fisik";
    }
}
```

Penjelasan:

Class PhysicalProduct merupakan *subclass* dari *class Product* yang digunakan untuk merepresentasikan produk fisik yang memiliki berat dan dapat dikirim. *Class* ini juga mengimplementasikan *interface Shippable* sehingga wajib menyediakan implementasi *method getWeightGrams()* untuk mengembalikan berat produk dalam satuan gram. Atribut *weightGrams* digunakan untuk menyimpan berat produk, sedangkan *constructor* menggunakan *super()* untuk menginisialisasi atribut dari *class* induk dan melakukan validasi agar berat produk lebih dari nol. Selain itu, *method category()* mengimplementasikan *method* abstrak dari *class Product* dengan

mengembalikan kategori *"Fisik"*. Dengan demikian, *class* ini menerapkan konsep *inheritance*, *interface*, dan *abstraction* dalam pemrograman berorientasi objek.

2) L2.b – *Interface PaymentMethod*

```
package shop.payment;

public interface PaymentMethod {
    String getName();

    /** @return true if the payment succedes */
    boolean pay(long amount);
}
```

Penjelasan:

Interface PaymentMethod merupakan *interface* yang digunakan sebagai aturan atau kontrak bagi berbagai jenis metode pembayaran dalam aplikasi toko. *Interface* ini memiliki dua *method*, yaitu *getName()* untuk mengambil nama metode pembayaran dan *pay(long amount)* untuk memproses pembayaran berdasarkan jumlah nominal yang harus dibayar. *Method pay()* menggunakan tipe data *boolean* untuk menunjukkan apakah pembayaran berhasil atau tidak. Setiap *class* yang mengimplementasikan *interface PaymentMethod* wajib menyediakan implementasi dari kedua *method* tersebut. Dengan demikian, *interface* ini menerapkan konsep *abstraction* karena hanya menentukan *method* yang harus tersedia tanpa menjelaskan detail proses pembayarannya.

3) L2.c – *Class EWalletPayment*

```
package shop.payment;

public class EWalletPayment implements PaymentMethod {
    private final String provider;
    private long balance;

    public EWalletPayment(String provider, long balance){
        if (provider == null || provider.isBlank()) {
```

```
            throw new IllegalArgumentException("Provider
cannot be null or blank");
        }
        if (balance < 0) {
            throw new IllegalArgumentException("Balance
cannot be negative");
        }
        this.provider = provider;
        this.balance = balance;
    }

    @Override
    public String getName() {
        return "E-Wallet " + provider;
    }
    public long getBalance() {
        return balance;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }
        balance -= amount;
        return true;
    }
}
```

Penjelasan:

Class EWalletPayment merupakan *class* yang mengimplementasikan *interface PaymentMethod* dan digunakan untuk merepresentasikan metode pembayaran menggunakan dompet digital atau *E-Wallet. Class* ini memiliki atribut *provider* untuk menyimpan nama penyedia layanan dompet digital dan balance untuk menyimpan jumlah saldo yang tersedia. *Constructor* digunakan untuk menginisialisasi kedua atribut tersebut serta melakukan validasi agar nama penyedia tidak kosong dan saldo tidak bernilai negatif. *Method getName()* digunakan untuk menampilkan nama metode pembayaran beserta penyedianya, sedangkan *method getBalance()* digunakan untuk mengambil jumlah saldo. Selain itu, *method pay()* digunakan

untuk memproses pembayaran dengan memeriksa apakah nominal pembayaran lebih dari nol dan tidak melebihi saldo yang tersedia. Jika pembayaran berhasil, saldo akan dikurangi sesuai nominal pembayaran dan *method* mengembalikan nilai *true*. Jika tidak memenuhi ketentuan, method mengembalikan nilai *false*. Dengan demikian, *class* ini menerapkan konsep *interface* dan *abstraction* dalam pemrograman berorientasi objek.

4) L2.d – *Class BankTransferPayment*

```
package shop.payment;

public class BankTransferPayment implements PaymentMethod{
    private final String bankName;

    public BankTransferPayment(String bankName) {
        if (bankName == null || bankName.isBlank()) {
            throw new IllegalArgumentException("Bank name
cannot be null or blank");
        }
        this.bankName = bankName;
    }

    @Override
    public String getName() {
        return "Transfer " + bankName;
    }

    @Override
    public boolean pay(long amount) {
        return amount > 0;
    }
}
```

Penjelasan:

Class BankTransferPayment merupakan *class* yang mengimplementasikan *interface PaymentMethod* dan digunakan untuk merepresentasikan metode pembayaran melalui transfer bank. *Class* ini memiliki atribut *bankName* untuk menyimpan nama bank yang digunakan. *Constructor* berfungsi untuk menginisialisasi nama bank serta melakukan validasi agar nama

bank tidak kosong. *Method getName()* digunakan untuk menampilkan nama metode pembayaran beserta nama bank, sedangkan *method pay()* digunakan untuk memeriksa nominal pembayaran. Jika nominal pembayaran lebih dari nol, *method* akan mengembalikan nilai *true*, sedangkan jika nominal bernilai nol atau negatif, akan mengembalikan nilai *false*. Dengan demikian, *class* ini menerapkan konsep *interface* dan *abstraction* dalam pemrograman berorientasi objek.

5) L2.e – *Class CheckoutService*

```
package shop.service;

import java.util.List;

import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.shipping.Shippable;
import shop.shipping.ShippingService;

public class CheckoutService {

    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
        }
        return total;
    }

    public int totalWeight(List<Product> items) {
        int grams = 0;
        for (Product product : items) {
            if (product instanceof Shippable shipabble) {
                grams += shipabble.getWeightGrams();
            }
        }
        return grams;
    }

    public long total(List<Product> items, ShippingService
shipping) {
```

```
        return subtotal(items) +
shipping.calculateCost(totalWeight(items));
    }

    public boolean checkout(List<Product> items,
ShippingService shipping, PaymentMethod payment) {
        if (items.isEmpty()) {
            return false;
        }
        return payment.pay(total(items, shipping));
    }
}
```

Penjelasan:

Class CheckoutService merupakan *class* yang digunakan untuk mengelola proses *checkout* atau penyelesaian pembelian produk. *Class* ini memiliki beberapa *method* utama, yaitu *subtotal()* untuk menghitung total harga seluruh produk berdasarkan harga akhir masing-masing produk, *totalWeight()* untuk menghitung total berat produk yang dapat dikirim dengan memeriksa apakah produk mengimplementasikan *interface Shippable*, dan *total()* untuk menghitung keseluruhan biaya pembelian dengan menambahkan subtotal produk dan biaya pengiriman. Selain itu, *method checkout()* digunakan untuk memproses pembayaran dengan memeriksa apakah daftar produk kosong. Jika daftar produk kosong, proses *checkout* akan dibatalkan dan mengembalikan nilai *false*. Jika terdapat produk, sistem akan menghitung total pembayaran dan memanggil *method pay()* dari metode pembayaran yang dipilih. Dengan demikian, class ini menggabungkan beberapa komponen *dari class Product, interface Shippable, PaymentMethod,* dan *class ShippingService* untuk menjalankan proses pembelian.

6) L2.f – *ShopApp*

```
package shop.app;

import java.util.List;
import shop.model.DigitalProduct;
```

```java
import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.shipping.RegularShipping;
import shop.shipping.ShippingService;

public class ShopApp {
    public static void main(String[] args) {
        // L1: abstract class and abstract method
        Product coffee = new PhysicalProduct("KOPI-001",
"Kopi Arabika Gayo 250g", 85_000, 300);
        Product rendang = new PhysicalProduct("RDG-002",
"Rendang Kemasan 250g", 120_000, 400);
        Product ebook = new DigitalProduct("EBK-003", "E-
book Resep Kue Nusantara", 50_000, "https://toko-
umkm.example/unduh/ebook-resep-kue-nusantara");

        List<Product> cart = List.of(coffee, rendang,
ebook);
        System.out.println("== Katalog ==");
        for (Product product : cart) {
            System.out.println(product.getInfo());
        }

        // L2: interface and program to an interface
        CheckoutService checkout = new CheckoutService();
        ShippingService regular = new RegularShipping();
        ShippingService express = new ExpressShipping();
        System.out.println("\n== Ringkasan ==");
        System.out.println("Subtotal      : Rp" +
checkout.subtotal(cart));
        System.out.println("Berat kiriman : " +
checkout.totalWeight(cart) + " g");
        System.out.println("Total Reguler : Rp" +
checkout.total(cart, regular));
        System.out.println("Total Express : Rp" +
checkout.total(cart, express));
    }
}
```

Penjelasan:

Class ShopApp merupakan *class* utama yang digunakan untuk menjalankan aplikasi toko melalui *method main()*. Pada *class* ini, dibuat beberapa objek produk, yaitu produk fisik berupa kopi dan

rendang serta produk digital berupa e-book dengan menggunakan *class PhysicalProduct* dan *DigitalProduct.* Seluruh produk kemudian dimasukkan ke dalam sebuah daftar menggunakan *List<Product>* yang berfungsi sebagai keranjang belanja. Program menampilkan katalog produk beserta informasi masing-masing produk melalui *method getInfo().* Selanjutnya, dibuat objek *CheckoutService* untuk mengelola perhitungan pembelian serta objek *RegularShipping* dan *ExpressShipping* untuk menyediakan pilihan layanan pengiriman. Program kemudian menampilkan subtotal harga produk, total berat kiriman, serta total biaya pembelian menggunakan layanan pengiriman reguler dan ekspres. Dengan demikian, *class* ini berfungsi sebagai titik awal untuk menghubungkan berbagai *class* yang telah dibuat dan menjalankan simulasi proses pembelian dalam aplikasi toko.

c. L3 – *Interface* lanjutan: *default, static, private method* dan *interface inheritance*

1) L3.a – *Class Discountable*

```
package shop.discount;

public interface Discountable {
    int discountPercent();

    default long applyDiscount(long price) {
        return price - percentOf(price, safePercent());
    }

    static long percentOf(long amount, int percent) {
        return amount * percent / 100;
    }

    private int safePercent() {
        int percent = discountPercent();
        if (percent < 0) {
            return 0;
        }
        return Math.min(percent, 100);
    }
}
```

Penjelasan:

Interface Discountable merupakan *interface* yang digunakan sebagai aturan bagi class yang memiliki fitur pemberian diskon pada produk. *Interface* ini memiliki *method* abstrak *discountPercent()* yang digunakan untuk menentukan persentase diskon. Selain itu, terdapat *default method applyDiscount()* untuk menghitung harga setelah diskon dengan memanggil *method percentOf()* dan *safePercent(). Method percentOf()* merupakan *static method* yang digunakan untuk menghitung nominal diskon berdasarkan harga dan persentase yang diberikan. Sementara itu, *private method safePercent()* berfungsi untuk memvalidasi persentase diskon agar tidak kurang dari 0% dan tidak lebih dari 100%. Dengan demikian, *interface* ini menerapkan konsep abstraction serta penggunaan *default method, static method,* dan *private method* pada *interface* Java.

2) L3.b – *Class DigitalProduct* ikut promo

```
package shop.model;

import shop.discount.Discountable;

public class DigitalProduct extends Product implements
Discountable{
    private static final int DIGITAL_DISCOUNT_PERCENT = 10;
    private final String downloadUrl;

    public DigitalProduct(String sku, String name, long
price, String downloadUrl) {
        super(sku, name, price);
        if (downloadUrl == null || downloadUrl.isBlank())
{
            throw new IllegalArgumentException("Download
URL must not be blank");
        }
        this.downloadUrl = downloadUrl;
    }

    public String getDownloadUrl() {
        return downloadUrl;
```

```
    }

    @Override
    public String category() {
        return "Digital";
    }

    @Override
    public int discountPercent() {
        return DIGITAL_DISCOUNT_PERCENT;
    }

    @Override
    public long getFinalPrice() {
        return applyDiscount(getPrice());
    }
}
```

Penjelasan:

Class DigitalProduct merupakan *subclass* dari *class Product* yang digunakan untuk merepresentasikan produk digital dan mengimplementasikan *interface Discountable* agar produk dapat memperoleh potongan harga. *Class* ini memiliki atribut *DIGITAL_DISCOUNT_PERCENT* yang bersifat *static final* untuk menetapkan persentase diskon sebesar 10% dan atribut *downloadUrl* untuk menyimpan alamat URL unduhan produk. *Constructor* menggunakan *super()* untuk menginisialisasi atribut dari class induk serta melakukan validasi agar URL unduhan tidak kosong. *Method getDownloadUrl()* digunakan untuk mengambil alamat URL unduhan, sedangkan *method category()* menentukan kategori produk sebagai *"Digital"*. *Method discountPercent()* mengembalikan persentase diskon sebesar 10% sesuai nilai yang telah ditetapkan. Sementara itu, *method getFinalPrice()* melakukan *override* terhadap *method* dari *class Product* untuk menghitung harga akhir setelah diskon dengan memanggil *applyDiscount()* dari *interface Discountable*. Dengan demikian, *class* ini menerapkan konsep *inheritance*, *interface*, *abstraction*, dan *method overriding*, serta memanfaatkan fitur diskon pada *interface* Java.

3) **L3.c – *Interface inheritance***

RefundablePayment

```
package shop.payment;

public interface RefundablePayment extends PaymentMethod {
    boolean refund(long amount);
}
```

Penjelasan:

Interface RefundablePayment merupakan *interface* turunan dari *interface PaymentMethod* yang digunakan untuk menambahkan fitur pengembalian dana atau *refund* pada metode pembayaran. Dengan menggunakan *extends, interface* ini mewarisi seluruh *method* yang dimiliki *PaymentMethod*, yaitu *getName()* dan *pay()*. Selain itu, *interface* ini menambahkan *method* abstrak *refund(long amount)* yang digunakan untuk memproses pengembalian dana berdasarkan nominal yang ditentukan. Setiap *class* yang mengimplementasikan *RefundablePayment* wajib menyediakan implementasi dari ketiga *method* tersebut. Dengan demikian, *interface* ini menerapkan konsep *interface inheritance* dan *abstraction* dalam pemrograman berorientasi objek.

InstallmentPayment

```
package shop.payment;

public interface InstallmentPayment extends PaymentMethod
{
     int maxInstallmentMonths();

     /** Monthly amount rounded up. */
     default long monthlyInstallment(long amount, int
months) {
        if (months < 1 || months > maxInstallmentMonths())
{
            throw new IllegalArgumentException("Months
must be between 1 and " + maxInstallmentMonths());
        }
        return (amount + months - 1) / months;
     }
}
```

Penjelasan:

Interface InstallmentPayment merupakan *interface* turunan dari *interface PaymentMethod* yang digunakan untuk menambahkan fitur pembayaran secara cicilan. Dengan menggunakan *extends, interface* ini mewarisi *method getName()* dan *pay()* dari *PaymentMethod.* Selain itu, terdapat *method* abstrak *maxInstallmentMonths()* yang digunakan untuk menentukan batas maksimal jumlah bulan cicilan. *Interface* ini juga memiliki *default method monthlyInstallment()* yang digunakan untuk menghitung nominal cicilan per bulan berdasarkan jumlah pembayaran dan lama cicilan. Sebelum melakukan perhitungan, *method* tersebut memeriksa apakah jumlah bulan cicilan berada dalam batas yang diizinkan. Hasil perhitungan dibulatkan ke atas agar nominal cicilan dapat dibagi menjadi bilangan bulat. Dengan demikian, *interface* ini menerapkan konsep *interface inheritance* dan *abstraction,* serta memanfaatkan fitur *default method* pada *interface* Java.

4) **L3.d – *Class CreditCardPayment***

Mengubah *Ewallet* menjadi *implements Refundable*

```
package shop.payment;

public class EWalletPayment implements RefundablePayment {
    private final String provider;
    private long balance;

    public EWalletPayment(String provider, long balance){
        if (provider == null || provider.isBlank()) {
            throw new IllegalArgumentException("Provider
cannot be null or blank");
        }
        if (balance < 0) {
            throw new IllegalArgumentException("Balance
cannot be negative");
        }
        this.provider = provider;
        this.balance = balance;
    }
```

```
    @Override
    public String getName() {
        return "E-Wallet " + provider;
    }
    public long getBalance() {
        return balance;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }
        balance -= amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0) {
            return false;
        }
        balance += amount;
        return true;
    }
}
```

Penjelasan:

Class EWalletPayment merupakan *class* yang mengimplementasikan *interface RefundablePayment* dan digunakan untuk merepresentasikan metode pembayaran menggunakan dompet digital atau *E-Wallet* yang memiliki fitur pengembalian dana. *Class* ini memiliki atribut *provider* untuk menyimpan nama penyedia layanan dan *balance* untuk menyimpan jumlah saldo pengguna. *Constructor* digunakan untuk menginisialisasi kedua atribut serta melakukan validasi agar nama penyedia tidak kosong dan saldo tidak bernilai negatif. *Method getName()* digunakan untuk menampilkan nama metode pembayaran beserta penyedianya, sedangkan *method getBalance()* digunakan untuk mengambil jumlah saldo. *Method pay()* digunakan untuk memproses pembayaran dengan memeriksa

apakah nominal pembayaran lebih dari nol dan tidak melebihi saldo yang tersedia. Jika pembayaran berhasil, saldo akan dikurangi sesuai nominal pembayaran. Sementara itu, *method refund()* digunakan untuk memproses pengembalian dana dengan memeriksa apakah nominal pengembalian lebih dari nol. Jika valid, saldo akan ditambahkan sesuai nominal pengembalian. Dengan demikian, *class* ini menerapkan konsep *interface*, *interface inheritance*, dan *abstraction* serta menyediakan fitur pembayaran dan pengembalian dana.

CreditCardPayment

```
package shop.payment;

public class CreditCardPayment implements
RefundablePayment, InstallmentPayment {
    private final String cardHolder;
    private final long creditLimit;
    private long usedCredit;

    public CreditCardPayment(String cardHolder, long
creditLimit) {
        if (cardHolder == null || cardHolder.isBlank()) {
            throw new IllegalArgumentException("Card
holder must not be blank");
        }
        if (creditLimit < 0) {
            throw new IllegalArgumentException("Credit
limit must not be negative");
        }
        this.cardHolder = cardHolder;
        this.creditLimit = creditLimit;
    }

    @Override
    public String getName() {
        return "Kartu Kredit (" + cardHolder + ")";
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || usedCredit + amount >
creditLimit) {
```

```
            return false;
        }
        usedCredit += amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0 || amount > usedCredit) {
            return false;
        }
        usedCredit -= amount;
        return true;
    }

    @Override
    public int maxInstallmentMonths() {
        return 12;
    }

    public long getUsedCredit() {
        return usedCredit;
    }
}
```

Penjelasan:

Class CreditCardPayment merupakan *class* yang mengimplementasikan dua *interface*, yaitu *RefundablePayment* dan *InstallmentPayment. Class* ini digunakan untuk merepresentasikan metode pembayaran menggunakan kartu kredit yang mendukung pembayaran, pengembalian dana, dan pembayaran secara cicilan. *Class* ini memiliki atribut *cardHolder* untuk menyimpan nama pemegang kartu, *creditLimit* untuk menentukan batas maksimal kredit, dan *usedCredit* untuk mencatat jumlah kredit yang telah digunakan. *Constructor* digunakan untuk menginisialisasi nama pemegang kartu dan batas kredit serta melakukan validasi agar data yang dimasukkan tidak kosong atau bernilai negatif. *Method getName()* digunakan untuk menampilkan nama metode pembayaran beserta nama pemegang kartu. *Method pay()* digunakan untuk memproses pembayaran dengan memastikan nominal lebih dari nol dan total kredit yang

digunakan tidak melebihi batas kredit. *Method refund()* digunakan untuk mengurangi jumlah kredit yang telah digunakan berdasarkan nominal pengembalian dana. Sementara itu, *method maxInstallmentMonths()* menentukan batas maksimal cicilan selama 12 bulan, sedangkan *getUsedCredit()* digunakan untuk mengambil jumlah kredit yang telah digunakan. Dengan demikian, *class* ini menerapkan konsep *multiple interface implementation*, *interface inheritance*, dan *abstraction* dalam pemrograman berorientasi objek.

CheckoutService

```
package shop.service;

import java.util.List;

import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.shipping.Shippable;
import shop.shipping.ShippingService;
import shop.payment.RefundablePayment;

public class CheckoutService {

    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
        }
        return total;
    }

    public int totalWeight(List<Product> items) {
        int grams = 0;
        for (Product product : items) {
            if (product instanceof Shippable shipabble) {
                grams += shipabble.getWeightGrams();
            }
        }
        return grams;
    }
```

```
    public long total(List<Product> items, ShippingService
shipping) {
        return subtotal(items) +
shipping.calculateCost(totalWeight(items));
    }

    public boolean checkout(List<Product> items,
ShippingService shipping, PaymentMethod payment) {
        if (items.isEmpty()) {
            return false;
        }
        return payment.pay(total(items, shipping));
    }

    public boolean refund(RefundablePayment payment, long
amount) {
        return payment.refund(amount);
    }
}
```

Penjelasan:

Class CheckoutService merupakan *class* yang digunakan untuk mengelola proses pembelian, perhitungan biaya, dan pengembalian dana dalam aplikasi toko. *Class* ini memiliki beberapa *method*, yaitu *subtotal()* untuk menghitung total harga seluruh produk berdasarkan harga akhir, *totalWeight()* untuk menghitung keseluruhan berat produk yang dapat dikirim dengan memeriksa apakah produk mengimplementasikan *interface Shippable*, dan *total()* untuk menghitung total pembayaran dengan menambahkan subtotal produk dan biaya pengiriman. *Method checkout()* digunakan untuk memproses pembayaran dengan memeriksa apakah keranjang belanja kosong. Jika keranjang kosong, proses pembayaran dibatalkan dan mengembalikan nilai *false*. Jika terdapat produk, sistem akan menghitung total biaya dan memproses pembayaran melalui *method pay()* dari *interface PaymentMethod*. Selain itu, terdapat *method refund()* yang digunakan untuk memproses pengembalian dana dengan memanggil *method refund()* dari *interface RefundablePayment*. Dengan demikian, class ini menggabungkan konsep *abstraction*

dan penggunaan beberapa *interface* untuk mengelola proses pembelian serta pengembalian dana.

5) L3.e – Lengkapi *ShopApp* dengan pembayaran

```
EWalletPayment wallet = new EWalletPayment("DompetKu",
300_000);
        PaymentMethod transfer = new
BankTransferPayment("Bank Nagari");
        System.out.println("\n== Pembayaran ==");
        System.out.println(wallet.getName() + " berhasil?
" + checkout.checkout(cart, regular, wallet));
        System.out.println("Sisa saldo: Rp" +
wallet.getBalance());
         System.out.println(transfer.getName() + " berhasil?
" + checkout.checkout(cart, express, transfer));

        // L3: interface inheritance, multiple interfaces,
default method
        CreditCardPayment card = new
CreditCardPayment("Siti Rahma", 1_000_000);
        System.out.println(card.getName() + " berhasil? "
+ checkout.checkout(cart, express, card));
        System.out.println("Cicilan 3 bulan: Rp" +
card.monthlyInstallment(checkout.total(cart, express), 3));

        System.out.println("\n== refund e-book ==");
        System.out.println("Refund e-wallet berhasil? " +
checkout.refund(wallet, ebook.getFinalPrice()));
        System.out.println("Saldo setelah refund: Rp" +
wallet.getBalance());
        // checkout.refund(transfer, 1_000); // compile
error: BankTransferPayment is not RefundablePayment
```

Penjelasan:

kode tersebut merupakan bagian dari *class ShopApp* yang digunakan untuk menjalankan simulasi proses pembayaran menggunakan beberapa metode pembayaran, yaitu *E-Wallet*, transfer bank, dan kartu kredit. Pertama, dibuat objek *EWalletPayment* dengan nama penyedia *"DompetKu"* dan saldo awal Rp300.000, serta objek *BankTransferPayment* dengan nama bank *"Bank Nagari"*. Program kemudian memproses pembayaran menggunakan layanan pengiriman reguler dan ekspres melalui

method checkout(), lalu menampilkan status keberhasilan pembayaran dan sisa saldo *e-wallet*. Selanjutnya, dibuat objek *CreditCardPayment* dengan nama pemegang kartu *"Siti Rahma"* dan batas kredit Rp1.000.000. Kartu kredit digunakan untuk melakukan pembayaran sekaligus menghitung nominal cicilan selama tiga bulan menggunakan *default method monthlyInstallment()*. Pada bagian akhir, program melakukan simulasi pengembalian dana (*refund*) untuk produk e-book melalui *e-wallet* dan menampilkan saldo setelah pengembalian dana. Baris kode yang diberi komentar menunjukkan bahwa proses refund menggunakan *BankTransferPayment* tidak dapat dilakukan karena *class* tersebut tidak mengimplementasikan *interface RefundablePayment*. Dengan demikian, kode ini menunjukkan penerapan konsep *interface*, *interface inheritance*, *multiple interface implementation*, dan *default method* dalam simulasi pembayaran dan pengembalian dana.

d. L4 – *Final method* dan *Final class*

1) L4.a – Kunci rumus ongkir

```
package shop.shipping;

public abstract class ShippingService {
    private final String name;
    protected ShippingService(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must
not be blank");
        }
        this.name = name;
    }

    public String getName() {
        return name;
    }

    /** Rate in rupiah per started kilogram. */
    protected abstract long ratePerKg();
```

```
    /** Weight is rounded up: 700 g -> 1 kg, 1001 g -> 2
kg. */
    public final long calculateCost(int weightGrams) {
        if (weightGrams <= 0) {
            return 0;
        }
        long kilograms = (weightGrams + 999L) / 1000;
        return kilograms * ratePerKg();
    }
}
```

Penjelasan:

Class ShippingService merupakan *abstract class* yang digunakan sebagai *class* induk untuk berbagai jenis layanan pengiriman. *Class* ini memiliki atribut name untuk menyimpan nama layanan pengiriman yang diinisialisasi melalui *constructor* dan divalidasi agar tidak kosong. *Method getName()* digunakan untuk mengambil nama layanan, sedangkan *method* abstrak *ratePerKg()* wajib diimplementasikan oleh setiap *subclass* untuk menentukan tarif pengiriman per kilogram. Selain itu, terdapat *method calculateCost()* yang digunakan untuk menghitung biaya pengiriman berdasarkan berat produk dalam gram. Perhitungan berat dilakukan dengan pembulatan ke atas, sehingga berat 700 gram dihitung sebagai 1 kilogram dan berat 1.001 gram dihitung sebagai 2 kilogram. *Method calculateCost()* menggunakan kata kunci *final* sehingga tidak dapat di-*override* oleh *subclass.* Dengan demikian, *class* ini menerapkan konsep *abstraction*, *inheritance*, dan penggunaan *final method* dalam pemrograman berorientasi objek.

2) L4.b – *Address immutable*

```
package shop.model;

public final class Address {
    private final String street;
    private final String city;
    private final String postalCode;
```

```
    public Address(String street, String city, String
postalCode) {
        if (street == null || street.isBlank()) {
            throw new IllegalArgumentException("Street
cannot be blank");
        }
        if (city == null || city.isBlank()) {
            throw new IllegalArgumentException("City
cannot be blank");
        }
        if (postalCode == null || postalCode.isBlank()) {
            throw new IllegalArgumentException("Postal
code cannot be blank");
        }
        this.street = street;
        this.city = city;
        this.postalCode = postalCode;
    }

    public String getStreet() {
        return street;
    }
    public String getCity() {
        return city;
    }
    public String getPostalCode() {
        return postalCode;
    }

    @Override
    public String toString() {
        return street + ", " + city + " " + postalCode;
    }
}
```

Penjelasan:

Class Address merupakan *class* yang digunakan untuk merepresentasikan alamat pengiriman yang terdiri dari tiga atribut, yaitu *street* untuk menyimpan nama jalan, *city* untuk menyimpan nama kota, dan *postalCode* untuk menyimpan kode pos. *Class* ini menggunakan kata kunci *final* sehingga tidak dapat diwariskan oleh class lain. Seluruh atribut juga menggunakan *final* agar nilainya tidak dapat diubah setelah objek dibuat. *Constructor* berfungsi untuk menginisialisasi ketiga atribut sekaligus

melakukan validasi agar tidak ada data alamat yang kosong. *Method getStreet(), getCity(),* dan *getPostalCode()* digunakan untuk mengambil nilai dari masing-masing atribut. Selain itu, *method toString()* melakukan *override* untuk menggabungkan seluruh informasi alamat menjadi satu teks yang mudah dibaca. Dengan demikian, *class* ini menerapkan konsep *encapsulation*, *immutability* (objek yang nilainya tidak dapat diubah setelah dibuat), dan penggunaan *final class* dalam pemrograman berorientasi objek.

e. L5 – *Nested class: static nested, inner,* dan *anonymous class*

1) L5.a – *Class Order*

```
package shop.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Order {
    private final String orderId;
    private final Address shippingAddress;
    private final List<Line> lines = new ArrayList<>();

    public Order(String orderId, Address shippingAddress)
{
        if (orderId == null || orderId.isBlank() ||
shippingAddress == null) {
            throw new IllegalArgumentException("Order id
and address are required");
        }
        this.orderId = orderId;
        this.shippingAddress = shippingAddress;
    }

    public String getOrderId() {
        return orderId;
    }
    public Address getShippingAddress() {
        return shippingAddress;
    }
```

```
    public void addLine(Product product, int quantity) {
        lines.add(new Line(product, quantity));
    }

    public List<Line> getLines() {
        return Collections.unmodifiableList(lines);
    }
```

Penjelasan:

Class Order merupakan *class* yang digunakan untuk merepresentasikan pesanan pelanggan dalam aplikasi toko. *Class* ini memiliki tiga atribut utama, yaitu *orderId* untuk menyimpan ID pesanan, *shippingAddress* untuk menyimpan alamat pengiriman, dan *lines* untuk menyimpan daftar produk beserta jumlah pembeliannya. *Constructor* digunakan untuk menginisialisasi ID pesanan dan alamat pengiriman serta melakukan validasi agar kedua data tersebut tidak kosong. *Method getOrderId()* dan *getShippingAddress()* digunakan untuk mengambil informasi pesanan dan alamat pengiriman. *Method addLine()* digunakan untuk menambahkan produk beserta jumlahnya ke dalam daftar pesanan dengan membuat objek *Line.* Sementara itu, *method getLines()* digunakan untuk menampilkan daftar produk yang dipesan dalam bentuk daftar yang tidak dapat dimodifikasi secara langsung dari luar class dengan memanfaatkan *Collections.unmodifiableList().* Dengan demikian, *class* ini menerapkan konsep *encapsulation*, *immutability* pada atribut tertentu, serta pengelolaan data pesanan menggunakan *ArrayList* dan *List.*

2) L5.b – Didalam *class order*

```
public long getItemsTotal() {
        long total = 0;
        for (Line line : lines) {
            total += line.subtotal();
        }
        return total;
    }
```

```
    public Receipt createReceipt() {
        return new Receipt();
    }

    // static nested: tidak butuh objek Order
    public static class Line {
        private final Product product;
        private final int quantity;

        public Line(Product product, int quantity) {
            if (product == null) {
                throw new
IllegalArgumentException("Product must not be null");
            }
            if (quantity <= 0) {
                throw new
IllegalArgumentException("Quantity must be positive");
            }
            this.product = product;
            this.quantity = quantity;
        }

        public Product getProduct() {
            return product;
        }
        public int getQuantity() {
            return quantity;
        }

        public long subtotal() {
            return product.getFinalPrice() * quantity;
        }
    }

    // inner class (tanpa static): selalu milik satu Order
    public class Receipt {
        public String toText() {
            StringBuilder sb = new StringBuilder();
            sb.append("\nSTRUK PESANAN
").append(Order.this.orderId).append("\n");
            for (Line line : lines) {
                sb.append("-
").append(line.getProduct().getName())
                  .append(" x").append(line.getQuantity())
                  .append(" =
Rp").append(line.subtotal()).append("\n");
            }
```

```
            sb.append("Total barang:
Rp").append(getItemsTotal()).append("\n");
            sb.append("Kirim ke:
").append(Order.this.shippingAddress);
            return sb.toString();
        }
    }
}
```

Penjelasan:

lanjutan dari *class Order* yang digunakan untuk mengelola perhitungan total pesanan dan pembuatan struk pembelian. *Method getItemsTotal()* digunakan untuk menghitung total harga seluruh produk dalam pesanan dengan menjumlahkan subtotal setiap produk berdasarkan harga akhir dan jumlah pembeliannya. *Method createReceipt()* digunakan untuk membuat objek *Receipt* sebagai struk pesanan. Di dalam *class Order* terdapat *static nested class Line* yang berfungsi menyimpan informasi produk dan jumlah pembelian, serta melakukan validasi agar produk tidak bernilai kosong dan jumlah pembelian harus lebih dari nol. *Method subtotal()* digunakan untuk menghitung harga produk berdasarkan jumlah yang dibeli. Selain itu, terdapat inner class Receipt yang digunakan untuk menghasilkan struk pesanan dalam bentuk teks melalui *method toText().* Struk tersebut menampilkan ID pesanan, nama produk, jumlah pembelian, subtotal setiap produk, total harga barang, dan alamat pengiriman. Penggunaan *Order.this* memungkinkan *inner class* mengakses atribut dan method dari objek *Order* yang menaunginya. Dengan demikian, kode ini menerapkan konsep *nested class, static nested class, inner class, encapsulation,* dan penggunaan *StringBuilder* untuk menyusun informasi struk secara dinamis.

3) L5.c – Lengkapi *ShopApp*

```
Address address = new Address("Jl. Moh Hatta No. 10",
"Padang", "25121");
        Order order = new Order("ORD-0001", address);
        order.addLine(coffee, 2);
```

```
        order.addLine(ebook, 1);

        Order.Receipt receipt = order.new Receipt(); //
inner object needs an outer object
        System.out.println(receipt.toText());

        PaymentMethod voucher = new PaymentMethod() { //
anonymous class
            private long remaining = 50_000;

            @Override
            public String getName() {
                return "Voucher Promo Hari Ini";
            }

            @Override
            public boolean pay(long amount) {
                if (amount <= 0 || amount > remaining) {
                    return false;
                }
                remaining -= amount;
                return true;
            }
        };

        System.out.println("\n" + voucher.getName() + "
untuk e-book berhasil? "
                + checkout.checkout(List.of(ebook),
regular, voucher));
    }
}
```

Penjelasan:

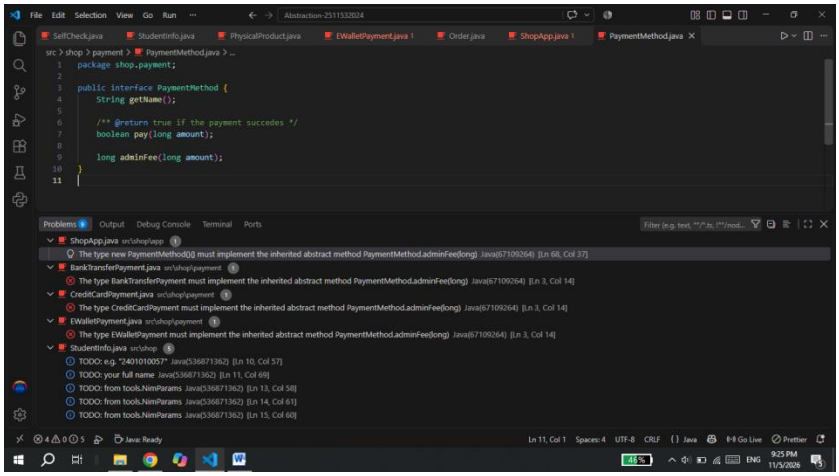
bagian dari *class ShopApp* yang digunakan untuk menjalankan simulasi pembuatan pesanan, pencetakan struk, dan pembayaran menggunakan voucher. Pertama, dibuat objek *Address* untuk menyimpan alamat pengiriman, kemudian objek *Order* dibuat dengan ID pesanan *"ORD-0001"* dan alamat tersebut. *Method addLine()* digunakan untuk menambahkan dua produk kopi dan satu produk e-book ke dalam pesanan. Selanjutnya, dibuat objek *Receipt* melalui *order.new Receipt()* karena *inner class* membutuhkan objek dari *class* luar (*Order*) untuk dapat dibuat. *Method toText()* kemudian dipanggil untuk menampilkan struk

yang berisi informasi pesanan. Pada bagian berikutnya, dibuat objek *PaymentMethod* menggunakan *anonymous class*, yaitu *class* tanpa nama yang langsung dibuat ketika objek dibutuhkan. Objek voucher memiliki atribut remaining untuk menyimpan saldo voucher sebesar Rp50.000, *method getName()* untuk menampilkan nama voucher, dan *method pay()* untuk memproses pembayaran dengan memeriksa nominal pembayaran serta sisa saldo voucher. Terakhir, program menjalankan proses pembayaran voucher untuk produk e-book menggunakan *method checkout()* dan menampilkan status keberhasilan transaksi. Dengan demikian, kode ini menerapkan konsep *inner class, object instantiation, anonymous class,* dan *interface implementation* dalam pemrograman berorientasi objek.

2. Tugas

a. T1 – Evolusi API dan konflik *diamond*

1) T1.a – Cara yang salah dulu.



Penjelasan:

"The type EWalletPayment must implement the inherited abstract method PaymentMethod.adminFee(long)"

class ini tidak *abstract*, tetapi masih ada *method abstract* warisan (*adminFee(long)* dari *PaymentMethod*) yang belum ada isinya. Java tidak mengizinkan itu.

- **Ada berapa berapa *error*, *class* mana saja, dan mengapa *anonymous class voucher* si *ShopApp* juga ikut rusak?**

  Ada 4 class yaitu: class *EWalletPayment*, *class CreditCardPayment*, *class BankTransferPayment*, dan *class ShopApp*. Alasan *Anonymous class voucher* di *ShopApp* ikut rusak karena dibuat dengan mengimplementasikan *PaymentMethod* secara langsung, tetapi belum menyediakan implementasi *adminFee()*.

2) **T1.b – *Default method***

```
default long adminFee(long amount) {
        return 0;
    }
```

Penjelasan:

Method adminFee() merupakan *default method* pada *interface* yang digunakan untuk menentukan biaya administrasi dari suatu transaksi berdasarkan nominal pembayaran. *Method* ini menerima parameter *amount* bertipe *long* dan mengembalikan nilai 0 sebagai biaya administrasi secara *default.* Karena menggunakan kata kunci *default*, *method* ini sudah memiliki implementasi sehingga *class* yang mengimplementasikan *interface* tersebut tidak wajib melakukan *override* terhadap *method* ini. Namun, setiap class dapat memberikan implementasi sendiri apabila memiliki ketentuan biaya administrasi yang berbeda. Dengan demikian, *method* ini memanfaatkan fitur *default method* pada *interface* Java untuk menyediakan implementasi bawaan yang dapat digunakan atau disesuaikan oleh *class* turunannya.

3) **T1.c – *Override* sesuai kebutuhan**

CreditCardPayment

```
import shop.StudentInfo;

 @Override
```

```
    public long adminFee(long amount) {
        return amount * StudentInfo.CARD_ADMIN_PERCENT /
100;
    }
```

Penjelasan:

kode tersebut digunakan untuk mengimplementasikan *method adminFee()* dengan melakukan *override* terhadap *default method* yang telah didefinisikan pada *interface. Import shop.StudentInfo* digunakan agar *class* dapat mengakses konstanta *CARD_ADMIN_PERCENT* yang menyimpan persentase biaya administrasi kartu. *Method adminFee()* menerima parameter amount bertipe *long*, kemudian menghitung biaya administrasi berdasarkan nominal transaksi dikalikan persentase biaya administrasi dan dibagi 100. Hasil perhitungan dikembalikan sebagai nilai biaya administrasi dalam bentuk bilangan bulat. Dengan demikian, kode ini menerapkan konsep *method overriding*, penggunaan *constant*, dan pemanfaatan *default method* pada *interface* Java.

BankTransferPayment

```
import shop.StudentInfo;

 @Override
    public long adminFee(long amount) {
        return StudentInfo.TRANSFER_ADMIN_FEE;
    }
```

Penjelasan:

kode tersebut digunakan untuk mengimplementasikan *method adminFee()* dengan melakukan *override* terhadap *default method* yang telah didefinisikan *pada interface. Import shop.StudentInfo* digunakan agar class dapat mengakses konstanta *TRANSFER_ADMIN_FEE* yang menyimpan nominal tetap biaya administrasi transfer. *Method adminFee()* menerima parameter amount bertipe *long*, tetapi tidak menggunakan nominal tersebut dalam perhitungan karena biaya administrasi ditentukan

berdasarkan nilai konstanta *TRANSFER_ADMIN_FEE*. Nilai tersebut kemudian dikembalikan sebagai biaya administrasi transaksi. Dengan demikian, kode ini menerapkan konsep *method overriding* dan penggunaan *constant* dalam pemrograman berorientasi objek.

4) T1.d – *CheckoutService*

```
public long grandTotal(List<Product> items,
ShippingService shipping, PaymentMethod payment) {
        long total = total(items, shipping);
        return total + payment.adminFee(total);
    }

public boolean checkout(List<Product> items,
ShippingService shipping, PaymentMethod payment) {
        if (items.isEmpty()) {
            return false;
        }
        return payment.pay(grandTotal(items, shipping,
payment));
    }
```

Penjelasan:

kode tersebut merupakan bagian dari *class CheckoutService* yang digunakan untuk menghitung total pembayaran termasuk biaya administrasi dan memproses transaksi pembelian. *Method grandTotal()* digunakan untuk menghitung keseluruhan biaya dengan menjumlahkan total harga produk dan biaya pengiriman melalui *method total()*, kemudian menambahkan biaya administrasi yang diperoleh dari *method adminFee()* pada *interface PaymentMethod*. Sementara itu, *method checkout()* digunakan untuk menjalankan proses pembayaran dengan memeriksa terlebih dahulu apakah daftar produk kosong. Jika keranjang belanja kosong, proses pembayaran dibatalkan dan mengembalikan nilai *false*. Jika terdapat produk, sistem akan menghitung total pembayaran beserta biaya administrasi, kemudian memproses pembayaran melalui *method pay()*. Hasil pembayaran akan mengembalikan nilai *true* jika berhasil dan *false*

jika gagal. Dengan demikian, kode ini menerapkan konsep *abstraction, polymorphism,* dan penggunaan *interface* untuk mendukung perhitungan biaya serta proses pembayaran dengan metode yang berbeda.

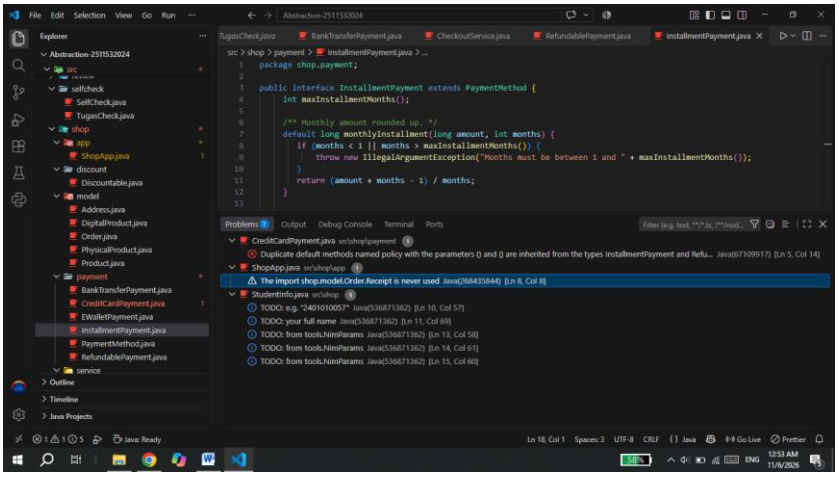
5) **T1.e – Konflik *diamond***

RefundablePayment

```
default String policy() {
        return "Refund maksimal 7 hari";
    }
```

InstallmentPayment

```
default String policy() {
        return "Cicilan maksimal " + maxInstallmentMonths()
+ " bulan";
    }
```



"*Duplicate default methods named policy with the parameters () and () are inherited from the types InstallmentPayment and RefundablePayment*"

CreditCardPayment

```
@Override
    public String policy() {
        return RefundablePayment.super.policy() + "; " +
InstallmentPayment.super.policy();
    }
```

Penjelasan:

Kode tersebut merupakan implementasi penyelesaian konflik diamond pada Java ketika sebuah kelas mengimplementasikan dua *interface* yang memiliki *default method* dengan nama dan parameter yang sama. *Method policy()* menggunakan anotasi *@Override* untuk menandakan bahwa kelas *CreditCardPayment* menyediakan implementasi sendiri dari *method* tersebut. Pada bagian *return, RefundablePayment.super.policy()* digunakan untuk memanggil implementasi *policy()* dari *interface RefundablePayment,* sedangkan *InstallmentPayment.super.policy()* digunakan untuk memanggil implementasi dari *interface InstallmentPayment.* Kedua hasil tersebut digabungkan menggunakan operator + dan dipisahkan dengan teks *"; "*. Dengan demikian, kedua kebijakan dapat ditampilkan sekaligus tanpa menimbulkan konflik. Hasilnya adalah "*Refund maksimal 7 hari; Cicilan maksimal 12 bulan*", dengan asumsi nilai maksimal cicilan adalah 12 bulan.

- **Mengapa *EWalletPayment* tidak perlu meng-*override policy()?***

  EWalletPayment hanya mengimplementasikan interface RefundablePayment yang memiliki satu implementasi default method policy(). Karena tidak terdapat konflik dengan default method dari interface lain, Java dapat langsung menggunakan implementasi bawaan dari RefundablePayment tanpa mengharuskan class tersebut melakukan override.

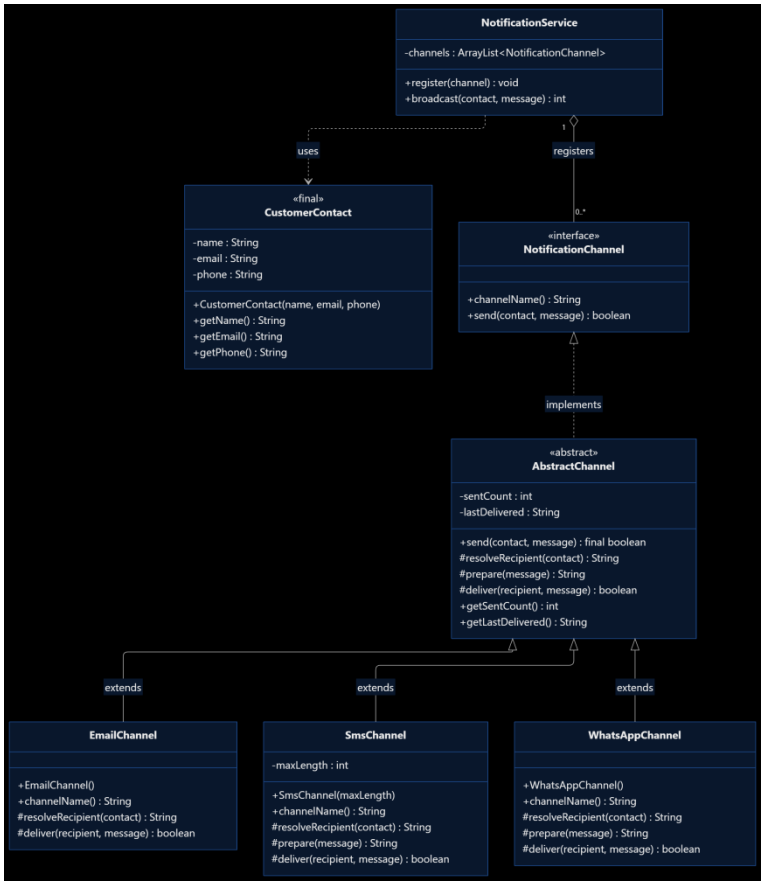
- **Mengapa java tidak memilih salah satu *default* secara otomatis?**

  Java tidak memilih salah satu implementasi secara otomatis karena kedua interface memiliki kedudukan yang sama dan menyediakan implementasi method policy() yang berbeda. Jika Java memilih salah satunya tanpa aturan yang jelas,

dapat terjadi ketidakpastian mengenai implementasi yang harus digunakan. Oleh karena itu, Java mengharuskan class yang mengalami konflik untuk melakukan override dan menentukan implementasi yang diinginkan secara eksplisit.

b. T2 – merancang sistem notifikasi pesanan

1) Rancangan UML



2) Implementasi UML

CustomerContact

```
package shop.notification;

public class CustomerContact {

    private final String name;
    private final String email;
    private final String phone;
```

```
    public CustomerContact(String name, String email,
String phone) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Nama wajib
diisi");
        }

        this.name = name;
        this.email = email;
        this.phone = phone;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public String getPhone() {
        return phone;
    }
}
```

Penjelasan:

CustomerContact menyimpan identitas dan informasi kontak pelanggan. Kata kunci *final* membuat kelas ini tidak dapat diwariskan, sedangkan atribut *final* memastikan nilainya tidak dapat diubah setelah objek dibuat. *Email* dan nomor telepon yang bernilai null otomatis diubah menjadi string kosong.

NotificationChannel

```
package shop.notification;

public class NotificationChannel {

    String channelName;

    boolean send(CustomerContact contact, String message);
}
```

Penjelasan:

Interface ini menjadi kontrak bagi seluruh kanal notifikasi. Setiap kanal wajib menyediakan nama kanal melalui *channelName()* dan kemampuan mengirim pesan melalui *send()*.

AbstractChannel

```
package shop.notification;

public abstract class AbstractChannel implements
NotificationChannel {

    private int sentCount = 0;
    private String lastDelivered = "";

    @Override
    public final boolean send(CustomerContact contact,
String message) {
        if (contact == null || message == null ||
message.isBlank()) {
            return false;
        }

        String recipient = resolveRecipient(contact);
        if (recipient == null || recipient.isBlank()) {
            return false;
        }

        String preparedMessage = prepare(message);

        boolean success = deliver(recipient,
preparedMessage);

        if (success) {
            sentCount++;
            lastDelivered = preparedMessage;
            return true;
        }
        return false;
    }

    protected abstract String
resolveRecipient(CustomerContact contact);
    protected abstract boolean deliver(String recipient,
String message);
```

```
    protected String prepare(String message) {
        return message;
    }

    public int getSentCount(){
        return sentCount;
    }

    public String getLAstDelivered() {
        return lastDelivered;
    }
}
```

Penjelasan:

Kelas abstrak *AbstractChannel* menjadi induk bagi kanal notifikasi. *Method send()* diberi kata kunci *final* agar alur pengiriman tidak dapat diubah oleh kelas turunannya. Alurnya dimulai dengan memvalidasi kontak dan pesan, mencari alamat tujuan melalui *resolveRecipient(),* menyiapkan pesan melalui *prepare(),* kemudian mengirimkannya melalui *deliver().* Jika pengiriman berhasil, jumlah pesan terkirim bertambah dan pesan terakhir yang berhasil dikirim disimpan. *Method resolveRecipient()* dan *deliver()* bersifat *abstract*, sehingga wajib diimplementasikan oleh setiap *subclass.*

EmailChannel

```
package shop.notification;

public class EmailChannel extends AbstractChannel {

    @Override
    public String channelName() {
        return "Email";
    }

    @Override
    protected String resolveRecipient(CustomerContact
contact) {
        String email = contact.getEmail();

        if (email.contains("@")) {
            return email;
```

```
        }
        return null;
    }

    @Override
    protected boolean deliver(String recipient, String
message) {
        System.out.println("[EMAIL] Kepada: " + recipient);
        System.out.println("Pesan: " + message);

        return true;
    }
}
```

Penjelasan:

EmailChannel merupakan *subclass* dari *AbstractChannel.* Kanal ini menggunakan alamat email sebagai tujuan pengiriman dan hanya menerima alamat yang mengandung karakter @. *Method deliver()* menyimulasikan pengiriman email dengan menampilkan alamat penerima dan isi pesan ke konsol.

SmsChannel

```
package shop.notification;

public class SmsChannel extends AbstractChannel {
    private final int maxLength;

    public SmsChannel(int maxLength) {
        if (maxLength < 3) {
            throw new IllegalArgumentException("Panjang
maksimum minimal 3 karakter");
        }

        this.maxLength = maxLength;
    }

    @Override
    public String channelName() {
        return "SMS";
    }

    @Override
    protected String resolveRecipient(CustomerContact
contact) {
```

```
        String phone = contact.getPhone();

        if (phone.startsWith("08")) {
            return phone;
        }
        return null;
    }

    @Override
    protected boolean deliver(String recipient, String
message) {
        System.out.println("[SMS] Kepada: " + recipient);
        System.out.println("Pesan: " + message);

        return true;
    }

    @Override
    protected String prepare(String message) {
        if (message.length() > maxLength) {
            return message.substring(0, maxLength - 3) +
"...";
        }
        return message;
    }

    @Override
    public String getLastDelivered() {
        return super.getLastDelivered();
    }

}
```

Penjelasan:

SmsChannel menerima nomor telepon yang diawali *08*. Atribut *maxLength* menentukan batas panjang pesan SMS. Jika pesan melebihi batas tersebut, *method prepare()* akan mengambil bagian awal pesan sebanyak *maxLength* - 3 karakter, kemudian menambahkan .... Dengan demikian, panjang pesan akhirnya tetap tepat sesuai batas yang ditentukan.

WhatsAppChannel

```
package shop.notification;
```

```
public class WhatsAppChannel extends AbstractChannel {

    @Override
    public String channelName() {
        return "WhatsApp";
    }

    @Override
    protected String resolveRecipient(CustomerContact
contact) {
        String phone = contact.getPhone();

        if ((phone.startsWith("08") ||
phone.startsWith("+62")) && phone.length() >= 10) {
            return phone;
        }
        return null;
    }

    @Override
    protected String prepare(String message) {
        return "[Toko UMKM] " + message;
    }

    @Override
    protected boolean deliver(String recipient, String
message) {
        System.out.println("[WHATSAPP] Kepada: " +
recipient);
        System.out.println("Pesan: " + message);

        return true;
    }
}
```

Penjelasan:

WhatsAppChannel menerima nomor telepon yang diawali *08* atau *+62* dengan panjang minimal 10 karakter. *Method prepare()* menambahkan awalan *[Toko UMKM]* pada setiap pesan sebelum dikirim. Proses pengiriman disimulasikan melalui tampilan konsol.

NotificationService

```
package shop.notification;
```

```java
import java.util.ArrayList;

public class NotificationService {
    private final ArrayList<NotificationChannel> channels
= new ArrayList<>();

    public void register(NotificationChannel channel) {
        if (channel != null) {
            channels.add(channel);
        }
    }

    public int broadcast(CustomerContact contact, String
message) {
        int successCount = 0;

        for (NotificationChannel channel : channels) {
            if (channel.send(contact, message)) {
                successCount++;
            }
        }

        return successCount;
    }
}
```

Penjelasan:

NotificationService bertugas mengelola dan mengirimkan notifikasi melalui seluruh kanal yang telah didaftarkan. Atribut *channels* menggunakan *ArrayList* bertipe *NotificationChannel*, sehingga dapat menampung berbagai objek kanal yang mengimplementasikan *interface* tersebut. *Method register()* digunakan untuk mendaftarkan kanal baru, sedangkan *broadcast()* mengirim pesan ke seluruh kanal dan mengembalikan jumlah pengiriman yang berhasil.

3) Demo

TugasApp

```java
package shop.app;

import java.util.List;
```

```java
import shop.model.Product;
import shop.model.PhysicalProduct;
import shop.StudentInfo;
import shop.model.DigitalProduct;

import shop.shipping.ExpressShipping;

import shop.payment.PaymentMethod;
import shop.payment.EWalletPayment;
import shop.payment.BankTransferPayment;
import shop.payment.CreditCardPayment;

import shop.service.CheckoutService;

import shop.notification.*;

public class TugasApp {

    public static void main(String[] args) {
        Product coffee = new PhysicalProduct("KOPI-001",
"Kopi Arabika Gayo 250g", 85_000, 300);
        Product rendang = new PhysicalProduct("RDG-002",
"Rendang Kemasan 250g", 120_000, 400);
        Product ebook = new DigitalProduct("EBK-003", "E-
book Resep Kue Nusantara", 50_000, "https://toko-
umkm.example/unduh/ebook-resep-kue-nusantara");

        List<Product> cart = List.of(coffee, rendang,
ebook);

        CheckoutService checkout = new CheckoutService();
        ExpressShipping express = new ExpressShipping();

        EWalletPayment wallet = new
EWalletPayment("DompetKu", 300_000);
        PaymentMethod transfer = new
BankTransferPayment("Bank Nagari");
        CreditCardPayment card = new
CreditCardPayment("Siti Rahma", 1_000_000);

        System.out.println("===== GRAND TOTAL =====");
        System.out.println("E-Wallet: Rp" +
checkout.grandTotal(cart, express, wallet));
        System.out.println("Bank Transfer: Rp" +
checkout.grandTotal(cart, express, transfer));
        System.out.println("Credit Card: Rp" +
checkout.grandTotal(cart, express, card));
```

```
        System.out.println("\n===== PAYMENT POLICY =====");
        System.out.println("Credit Card: " +
card.policy());
        System.out.println("E-Wallet: " + wallet.policy());

        NotificationService service = new
NotificationService();

        service.register(new EmailChannel());
        service.register(new
SmsChannel(StudentInfo.SMS_MAX_LENGTH));
        service.register(new WhatsAppChannel());
        service.register(new NotificationChannel() {

            @Override
            public String channelName() {
                return "Console Log";
            }

            @Override
            public boolean send(CustomerContact contact,
String message) {
                System.out.println("[LOG] " +
contact.getName() + ": " + message);
                return true;
            }
        });

        String message = "Pesanan Anda telah berubah
status menjadi "
            + "sedang diproses. Tim Toko UMKM sedang
menyiapkan "
            + "pesanan Anda dengan teliti agar seluruh
produk sesuai "
            + "dengan permintaan. Setelah proses persiapan
selesai, "
            + "pesanan akan segera diserahkan kepada kurir
untuk "
            + "dikirimkan ke alamat tujuan. Terima kasih
telah "
            + "berbelanja di Toko UMKM. Kami akan
memberikan "
            + "informasi kembali apabila terdapat
perubahan status "
            + "pesanan berikutnya.";
```

```
        CustomerContact andi = new CustomerContact("Andi",
"andi@gmail.com", "081234567890");
        CustomerContact budi = new CustomerContact("Budi",
"", "+6281234567890");

        System.out.println("\n===== NOTIFIKASI ANDI
=====");
        System.out.println("Berhasil: " +
service.broadcast(andi, message) + " kanal");

        System.out.println("\n===== NOTIFIKASI BUDI
=====");
        System.out.println("Berhasil: " +
service.broadcast(budi, message) + " kanal");
    }
}
```

Penjelasan:

Program *TugasApp* merupakan kelas utama yang digunakan untuk menguji implementasi sistem toko dan sistem notifikasi pesanan. Pada bagian awal, program membuat tiga objek produk, yaitu kopi dan rendang menggunakan kelas *PhysicalProduct*, serta e-book menggunakan kelas *DigitalProduct.* Ketiga produk tersebut dimasukkan ke dalam *List* bernama *cart* sebagai keranjang belanja. Selanjutnya, program membuat objek *CheckoutService* untuk mengelola perhitungan pembayaran dan *ExpressShipping* sebagai layanan pengiriman ekspres. Program juga membuat tiga metode pembayaran, yaitu *E-Wallet*, *Bank Transfer*, dan *Credit Card*. *Method grandTotal()* digunakan untuk menghitung total belanja beserta biaya pengiriman dan biaya administrasi masing-masing metode pembayaran. Setelah itu, program menampilkan hasil perhitungan ketiga metode pembayaran serta informasi kebijakan pembayaran menggunakan *method policy()* pada objek kartu kredit dan e-wallet.

Pada bagian berikutnya, program membuat objek *NotificationService* untuk mengelola pengiriman notifikasi. *Method register()* digunakan untuk mendaftarkan tiga kanal notifikasi, yaitu *EmailChannel*, *SmsChannel*, dan

WhatsAppChannel. Kanal SMS menggunakan konstanta *StudentInfo.SMS_MAX_LENGTH* sebagai batas maksimal panjang pesan. Selain itu, program membuat *anonymous class* yang mengimplementasikan *interface NotificationChannel* secara langsung untuk menyediakan kanal *Console Log* tanpa membuat kelas terpisah. Program kemudian membuat variabel *message* yang berisi informasi perubahan status pesanan dengan panjang lebih dari 200 karakter. Dua objek *CustomerContact,* yaitu *Andi* dan *Budi,* dibuat dengan informasi kontak yang berbeda untuk menguji kemampuan setiap kanal dalam menentukan alamat tujuan pengiriman. Terakhir, *method broadcast()* digunakan untuk mengirimkan pesan kepada kedua pelanggan melalui seluruh kanal yang telah didaftarkan dan mengembalikan jumlah kanal yang berhasil mengirimkan notifikasi. Andi menerima notifikasi melalui empat kanal, sedangkan Budi hanya menerima notifikasi melalui *WhatsApp* dan *Console Log* karena tidak memiliki alamat email dan nomor teleponnya menggunakan awalan *+62,* sehingga tidak memenuhi ketentuan penerima email maupun SMS.

- **Mengapa ada *interface* dan *abstract class,* bukan salah satunya saja?**

  Interface NotificationChannel digunakan sebagai kontrak yang menentukan bahwa setiap kanal notifikasi harus memiliki *method channelName()* dan *send().* Sementara itu, *abstract class AbstractChannel* digunakan untuk menyediakan implementasi umum berupa alur pengiriman pesan, validasi pesan, pencarian alamat tujuan, persiapan pesan, dan pencatatan hasil pengiriman. Keduanya memiliki fungsi berbeda dan saling melengkapi. Dengan demikian, *interface* memberikan fleksibilitas dalam penerapan kanal, sedangkan *abstract class* mengurangi pengulangan kode melalui *inheritance.*

- **Apa akibatnya jika send(…) tidak dibuat final?**
  Jika *method send()* tidak dibuat *final*, kelas turunannya dapat melakukan *override* terhadap *method* tersebut dan mengubah alur pengiriman yang sudah ditentukan oleh *AbstractChannel.* Akibatnya, setiap kanal dapat memiliki proses validasi, persiapan pesan, dan pencatatan yang berbeda-beda. Hal ini dapat menyebabkan ketidakkonsistenan perilaku antar kanal. Dengan menggunakan *final,* alur utama pengiriman tetap terjaga, sedangkan setiap kanal hanya perlu menyesuaikan bagian tertentu melalui *method* seperti *resolveRecipient(), prepare(),* dan *deliver().*
- **Mengapa *NotificationService* menyimpan *List<NotificationChannel>,* bukan *List<AbstractChannel>?***
  Karena *NotificationChannel* merupakan *interface* yang menjadi kontrak umum bagi seluruh kanal notifikasi. Dengan menggunakan *List<NotificationChannel>*, *NotificationService* dapat menerima berbagai objek yang mengimplementasikan *interface* tersebut, termasuk EmailChannel, SmsChannel, WhatsAppChannel, maupun *anonymous class* untuk *Console Log.* Jika menggunakan *List<AbstractChannel>*, objek yang mengimplementasikan *NotificationChannel* secara langsung tanpa mewarisi *AbstractChannel* tidak dapat didaftarkan. Oleh karena itu, penggunaan *interface* membuat sistem lebih fleksibel dan mendukung penerapan konsep *polymorphism.*

c. T3 – Code *review BadShop*

No	Lokasi (class/method)	masalah	Dampak nyata (bug atau risiko)	Perbaikan yang diusulkan
1	*StoreOperations*	*Interface* menggabungkan terlalu banyak	*Class* yang tidak membutuhkan semua operasi tetap harus	Memisahkan *interface* menjadi *Payable,*

		operasi, yaitu pembayaran, *refund*, pengiriman, dan pencetakan *invoice.*	mengimplementasikan seluruh *method.*	*Refundable, Shippable,* dan *InvoicePrintable.*
2	*GiftVoucher.refund( )*	*Method refund* melempar *UnsupportedOperationException.*	Program mengalami *exception* jika *voucher* digunakan untuk melakukan *refund.*	Tidak mengimplementasikan *interface Refundable* pada *GiftVoucher* karena *voucher* tidak mendukung *refund*.
3	*GiftVoucher.ship()*	*Voucher* dipaksa mengimplementasikan *method* pengiriman meskipun bukan barang fisik.	Program berhenti dengan *UnsupportedOperationException* ketika *method ship()* dipanggil.	Memisahkan kemampuan pengiriman menggunakan *interface Shippable* dan hanya menerapkannya pada barang fisik.
4	*Courier.cost()*	Perhitungan ongkir menggunakan parameter *String* dan *switch*, bukan implementasi perilaku melalui *subclass.*	Kesalahan penulisan jenis layanan seperti "EXPRES" menghasilkan ongkir 0 tanpa pemberitahuan.	Menggunakan *abstract method calculateCost()* yang diimplementasikan oleh setiap *subclass* kurir.
5	*Address*	*Class* belum menggunakan *final*, atribut dapat diubah, dan tersedia *setter*.	Alamat yang digunakan bersama oleh beberapa pesanan dapat berubah secara tidak sengaja.	Menjadikan *Address* sebagai *final class,* menggunakan atribut *private*

				final, dan menghapus *setter*.
6	*SimpleOrder*	Menyimpan referensi objek *Address* secara langsung dan menyediakan akses ke objek tersebut melalui *getter*.	Perubahan objek alamat dari luar dapat memengaruhi informasi pesanan.	Menggunakan objek *Address immutable* agar alamat pesanan tidak dapat diubah setelah dibuat.
7	*Discount.apply()*	*Default method* menggunakan *instanceof* untuk membedakan jenis diskon.	Penambahan jenis diskon baru mengharuskan perubahan pada *method* yang sudah ada.	Mengubah *apply()* menjadi *method interface* yang diimplementasikan masing-masing *class* diskon.
8	*Cart.Item*	Menggunakan *inner class non-static* meskipun tidak membutuhkan akses ke objek *Cart.*	Pembuatan objek Item harus melalui *instance Cart*, sehingga ketergantungan antarkelas tidak diperlukan.	Mengubah Item menjadi *static nested class.*

BadShopRefactored

```
package review.fixed;

import java.util.ArrayList;
import java.util.List;

interface Payable {
    boolean pay(long amount);
}

interface Refundable {
    boolean refund(long amount);
}
```

```java
interface Shippable {
    long ship(int weightGrams);
}

interface InvoicePrintable {
    String printInvoice();
}

final class GiftVoucher implements Payable,
InvoicePrintable {
    private double balance = 100000.0;

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }

        balance -= amount;
        return true;
    }

    @Override
    public String printInvoice() {
        return "sisa Rp" + (long) balance;
    }
}

abstract class Courier {
    public abstract long calculateCost(int kilograms);
}

class JneCourier extends Courier {
    @Override
    public long calculateCost(int kilograms) {
        if (kilograms <= 0) {
            throw new IllegalArgumentException("Berat
harus lebih dari 0");
        }

        return kilograms * 18000L;
    }
}

class Cart {
    private final List<Item> items = new ArrayList<>();
```

```
    static class Item {
        private final String name;
        private final long price;
        private final int quantity;

        Item(String name, long price, int quantity) {
            this.name = name;
            this.price = price;
            this.quantity = quantity;
        }

        long subtotal() {
            return price * quantity;
        }

        String getName() {
            return name;
        }
    }

    void add(Item item) {
        items.add(item);
    }

    long total() {
        long total = 0;

        for (Item item : items) {
            total += item.subtotal();
        }

        return total;
    }
}

final class Address {
    private final String street;
    private final String city;

    Address(String street, String city) {
        this.street = street;
        this.city = city;
    }

    @Override
    public String toString() {
```

```
        return street + ", " + city;
    }
}

final class SimpleOrder {
    private final String id;
    private final Address address;

    SimpleOrder(String id, Address address) {
        if (address == null) {
            throw new IllegalArgumentException("Alamat
tidak boleh null");
        }

        this.id = id;
        this.address = address;
    }

    Address getAddress() {
        return address;
    }

    @Override
    public String toString() {
        return id + " -> " + address;
    }
}

interface Discount {
    long apply(long price);
}

class MemberDiscount implements Discount {
    @Override
    public long apply(long price) {
        return price - price * 5 / 100;
    }
}

class FlashSaleDiscount implements Discount {
    @Override
    public long apply(long price) {
        return price - price * 30 / 100;
    }
}

public class BadShopRefactored {
```

```
    public static void main(String[] args) {

        Cart cart = new Cart();
        cart.add(new Cart.Item("Kopi Gayo", 85000, 2));
        System.out.println("Total cart: " + cart.total());

        Courier courier = new JneCourier();
        System.out.println("Ongkir Ekspres 2 kg: " +
courier.calculateCost(2));

        Address home = new Address("Jl. Sudirman 10",
"Padang");
        Address otherAddress = new Address("Jl. Sudirman
10", "Bukittinggi");

         SimpleOrder first = new SimpleOrder("ORD-1", home);
        SimpleOrder second = new SimpleOrder("ORD-2",
otherAddress);

        System.out.println(first);
        System.out.println(second);

        Discount flashSale = new FlashSaleDiscount();
        System.out.println("Harga flash sale: " +
flashSale.apply(100000));

        GiftVoucher voucher = new GiftVoucher();
        System.out.println(voucher.pay(30000) + " | " +
voucher.printInvoice());
    }
}
```

d. T4 – Refleksi: kapan memakai apa

1) Tabel 1 – *Abstract class vs interface*

Pertanyaan desain	Abstract class atau interface	Alasan	Contoh dari proyek
Subclass berbagi *field* dan konstruktor	*Abstract class*	*Abstract class* dapat menyimpan atribut dan menyediakan konstruktor yang diwariskan oleh	*Product* sebagai *abstract class* yang memiliki atribut name, price, dan *id*, kemudian diwariskan

		subclass.	oleh *PhysicalProduct* dan *DigitalProduct.*
Satu *class* perlu beberapa kemampuan sekaligus	*Interface*	Satu *class* dapat mengimplementasikan beberapa *interface* sekaligus sehingga bisa memiliki berbagai kemampuan.	*GiftVoucher* mengimplementasikan *Payable* dan *InvoicePrintable* untuk melakukan pembayaran dan mencetak *invoice.*
Rumus bersama yang tidak boleh diubah *subclass*	*Abstract class* dengan *method final*	*Method final* dapat menyediakan perhitungan yang sama dan mencegah subclass mengubah implementasinya.	*CheckoutService* dapat memiliki method perhitungan total yang dibuat final agar rumus perhitungan total tidak diubah oleh subclass jika nantinya dibuat pewarisan.
Kontrak yang perlu ditambah method baru di kemudian hari	*Interface*	*Interface* memudahkan penambahan kontrak kemampuan baru yang dapat diimplementasikan oleh berbagai *class* tanpa harus berada dalam satu hierarki pewarisan.	*NotificationChannel* memiliki *method send()* dan *channelName().* Nantinya dapat ditambahkan *method* baru atau *interface* kanal notifikasi untuk Telegram dan *push notification.*

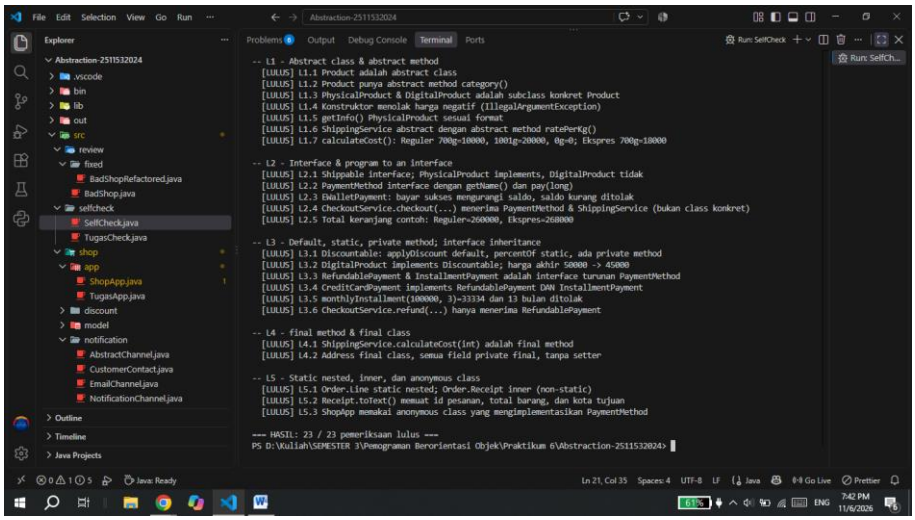
2) Tabel 2 – Jenis *nested class*

kondisi	Static nested, inner, atau anonymous?	Alasan	Contoh dari proyek

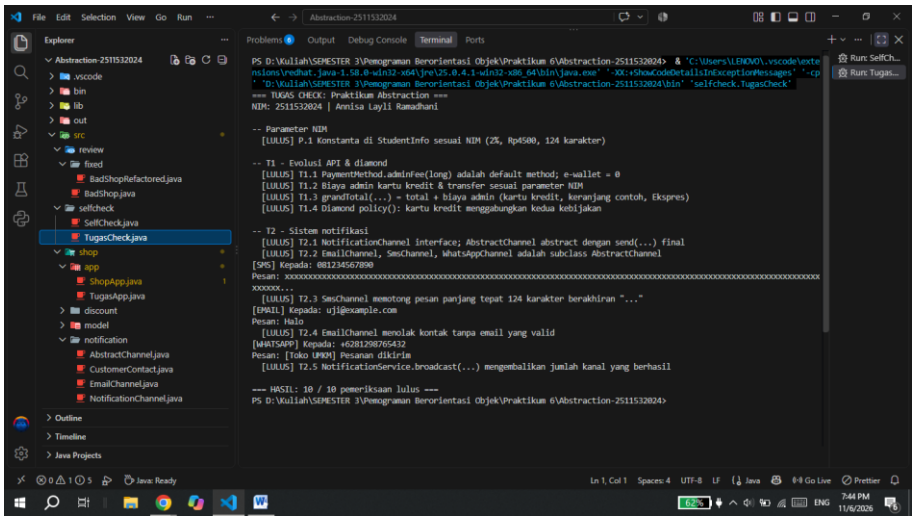
Class pembantu tidak membaca data objek luar	*Static nested class*	*Class* dapat dibuat tanpa harus memiliki *instance* dari *class* luarnya karena tidak membutuhkan akses ke atribut objek luar.	*Cart.Item* digunakan untuk menyimpan nama produk, harga, dan jumlah barang tanpa harus membaca atribut objek *Cart* secara langsung.
Objek harus membaca *field* objek luarnya	*Inner class*	*Inner class* dapat mengakses atribut dan *method* milik *instance class* luar secara langsung.	*Order.Receipt* dapat mengakses nomor pesanan, alamat, dan daftar item milik objek *Order* yang membuatnya.
Implementasi sekali pakai, tidak perlu nama	*Anonymous class*	*Anonymous class* cocok untuk implementasi *interface* atau *subclass* yang hanya digunakan satu kali tanpa perlu membuat *class* terpisah.	Implementasi *PaymentMethod* secara langsung di *ShopApp* untuk membuat pembayaran menggunakan *voucher* promo.

D. Tangkapan Layar Tugascheck, Selfcheck, Dan Keluaran TugasApp

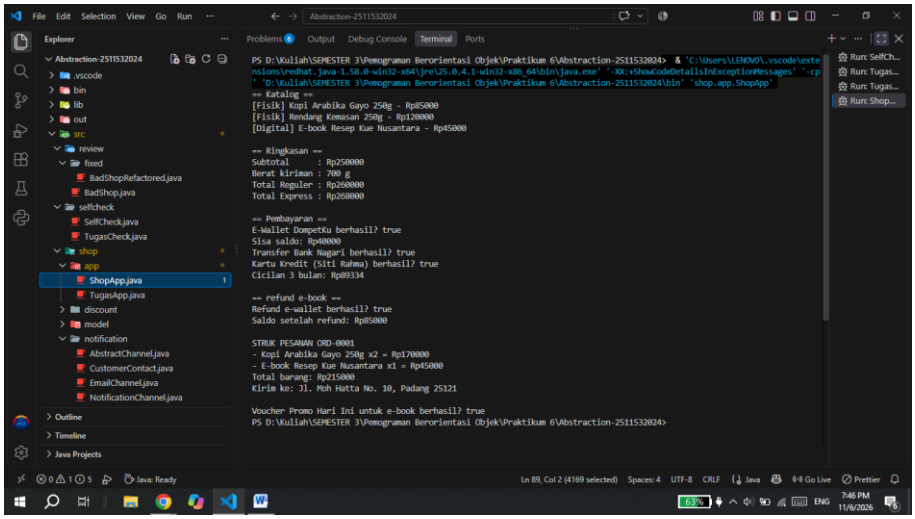
1. TugasCheck



2. SelfCheck


PS D:\Kuliah\SEMESTER 3\Pemograman Berorientasi Objek\Praktikum 6\Abstraction-2511532024> & 'C:\Users\LENOVO\.vscode\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'D:\Kuliah\SEMESTER 3\Pemograman Berorientasi Objek\Praktikum 6\Abstraction-2511532024\bin' 'selfcheck.TugasCheck'
=== TUGAS CHECK: Praktikum Abstraction ===
NIM: 2511532024 | Annisa Layli Ramadhani
-- Parameter NIM
[LULUS] P.1 Konstanta di StudentInfo sesuai NIM (2%, Rp4500, 124 karakter)
-- T1 - Evolusi API & diamond
[LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method; e-wallet = 0
[LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
[LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh, Ekspres)
[LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan
-- T2 - Sistem notifikasi
[LULUS] T2.1 NotificationChannel interface; AbstractChannel abstract dengan send(...) final
[LULUS] T2.2 EmailChannel, SmsChannel, WhatsAppChannel adalah subclass AbstractChannel
[SMS] Kepada: 081234567890
Pesan: xxxx

4. Output ShopApp



5. Output BadshopRefactored

